



دانشگاه علم و صنعت

دانشکده مهندسی کامپیوتر

داکیومنت فنی و پیاده‌سازی Backend

سامانه هوشمند عامل‌گرا برای تحلیل متون
پزشکی

(Agentic Medical NLP System)

نگارش:

امیرحسین امجدیان

اساتید محترم:

جناب آقای دکتر اعتمادی
جناب آقای دکتر مینایی

درس پردازش زبان‌های طبیعی (NLP)

نیمسال اول ۱۴۰۴-۱۴۰۵

فهرست مطالب

۲	۱	مقدمه و کلیات
۲	۱.۱	تعریف مسئله
۲	۲.۱	راه‌حل پیشنهادی: سیستم عاملی هوشمند
۲	۳.۱	نوآوری‌های پروژه
۳	۲	معماری سیستم
۳	۱.۲	نمای کلی سیستم
۴	۲.۲	توصیف ماژول‌های اصلی
۴	۱.۲.۲	لایه داده (Data Layer)
۴	۲.۲.۲	لایه مدل (Model Factory)
۴	۳.۲.۲	هسته عامل (Agent Core)
۴	۳	روش‌های تنظیم دقیق مدل (Fine-Tuning Strategies)
۵	۱.۳	مدل‌های پایه
۵	۲.۳	روش LoRA (Low-Rank Adaptation)
۵	۳.۳	روش Adapter
۶	۴.۳	مقایسه پارامترها
۶	۴	چرخه حیات عامل هوشمند (Agentic Workflow)
۶	۱.۴	تصمیم‌گیری خودکار (Orchestrator)
۶	۲.۴	مدیریت حافظه (Experience Memory)
۶	۳.۴	مدیریت بودجه (Budget Manager)
۷	۵	مجموعه داده و پیش‌پردازش
۷	۶	تفسیرپذیری و ارزیابی (Evaluation & XAI)
۷	۱.۶	معیارهای ارزیابی
۸	۲.۶	تفسیرپذیری (Explainable AI)
۸	۱.۲.۶	روش Layer Integrated Gradients
۸	۲.۲.۶	توضیحات بالینی هوشمند
۸	۳.۶	نمودارهای تحلیل عملکرد
۸	۷	پیاده‌سازی و رابط کاربری (Implementation & UI)
۸	۱.۷	تکنولوژی‌های مورد استفاده
۸	۲.۷	راهنمای کاربری سامانه
۹	۱.۲.۷	حالت آموزش (Training Mode)
۹	۲.۲.۷	حالت استنتاج (Inference Mode)

۱ مقدمه و کلیات

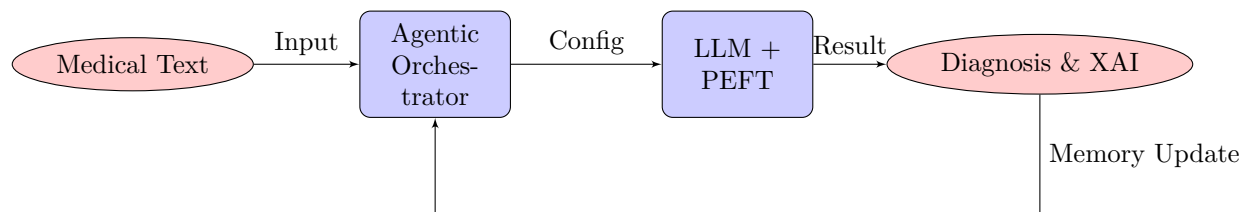
۱.۱ تعریف مسئله

امروزه طبقه‌بندی متون پزشکی (Medical Text Classification) یکی از چالش‌های بنیادین در حوزه پردازش زبان طبیعی (NLP) محسوب می‌شود. در این پروژه، هدف اصلی تشخیص خودکار دسته‌های بیماری یا شرایط بالینی از روی متن‌های خام پزشکی است. این مسئله با دو چالش عمده روبرو است:

- **کمبود داده:** دسترسی به داده‌های برجسب‌خورده پزشکی دشوار است و مدل‌ها باید با تعداد نمونه‌های کم (مثلاً کمتر از 1000 نمونه) آموزش ببینند.
- **نیاز به دقت بالا:** در حوزه پزشکی، اشتباهات مدل می‌تواند هزینه بالایی داشته باشد، بنابراین نیاز است که میزان عدم قطعیت (Uncertainty) مدل محاسبه شود.

۲.۱ راه‌حل پیشنهادی: سیستم عاملی هوشمند

برای غلبه بر چالش‌های فوق، ما یک Agentic System طراحی کرده‌ایم. این سیستم به جای اینکه صرفاً یک مدل طبقه‌بندی‌کننده باشد، یک عامل هوشمند است که فرآیند آموزش و ارزیابی را مدیریت می‌کند. در این سیستم، یک Orchestrator وجود دارد که با استفاده از مدل‌های زبانی بزرگ (مانند Gemini یا Llama)، بهترین تنظیمات را برای آموزش پیشنهاد می‌دهد و نتایج را در حافظه تجربیات (Experience Memory) ذخیره می‌کند. برای درک بهتر جریان کاری سیستم پیشنهادی، نمودار زیر معماری کلی تعاملات را نشان می‌دهد:



شکل ۱: نمای کلی جریان داده در سیستم عاملی پیشنهادی

۳.۱ نوآوری‌های پروژه

این پژوهش سه نوآوری اصلی را برای بهبود کارایی و قابلیت اطمینان مدل‌های زبانی در حوزه پزشکی ارائه می‌دهد:

۱. **تنظیم دقیق کارآمد پارامتر (PEFT):** به جای آموزش کامل مدل که هزینه‌بر است، از تکنیک‌هایی مانند LoRA و Adapter استفاده شده است. این روش‌ها تعداد پارامترهای قابل آموزش را به شدت کاهش می‌دهند (گاهی کمتر از 1% کل پارامترها).
۲. **تفسیرپذیری مدل (XAI):** با استفاده از روش Integrated Gradients، کلمات کلیدی که باعث تصمیم‌گیری مدل شده‌اند استخراج می‌شوند. این امر به پزشکان اجازه می‌دهد تا دلیل تشخیص مدل را درک کنند.

۳. مدیریت آزمایش‌ها توسط LLM Orchestrator: یک عامل هوشمند با بررسی تاریخچه آزمایش‌های قبلی، هاپیرپارامترهای آزمایش جدید (مانند نرخ یادگیری و تعداد Epoch) را پیشنهاد می‌دهد تا با کمترین تعداد آزمایش، به بیشترین دقت برسیم.

در جدول زیر، مقایسه‌ای بین روش‌های سنتی و روش پیشنهادی در این پروژه آورده شده است:

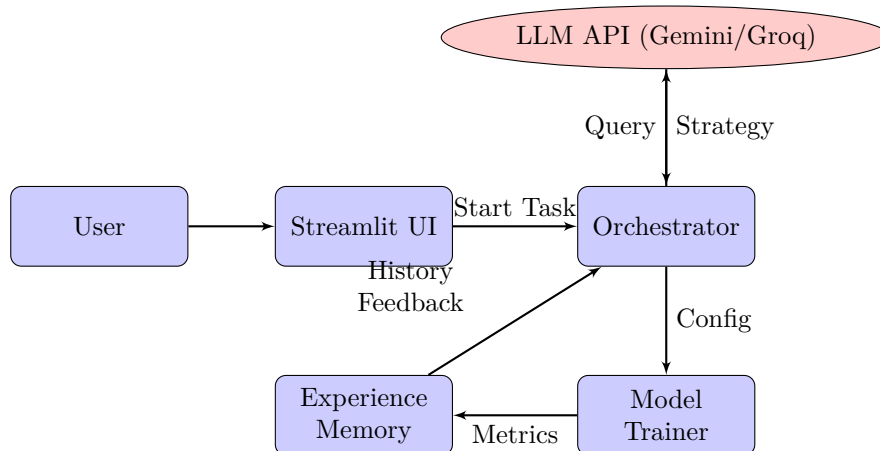
جدول ۱: مقایسه روش پیشنهادی با روش‌های سنتی

ویژگی	روش سنتی (Fine-Tuning)	روش پیشنهادی (Agentic PEFT)
مدیریت آموزش	دستی توسط کاربر	خودکار توسط Orchestrator
تعداد پارامترها	100% کل شبکه	کمتر از 1% (با LoRA)
تفسیرپذیری	جعبه سیاه (Black Box)	شفاف (XAI Report)
مدیریت حافظه	ندارد	دارد (Experience Memory)

۲ معماری سیستم

۱.۲ نمای کلی سیستم

معماری این سامانه بر اساس الگوی طراحی Modular Agentic Design بنا شده است. در این معماری، کاربر از طریق یک رابط گرافیکی تعاملی (Streamlit) با سیستم ارتباط برقرار می‌کند و تمامی درخواست‌ها توسط یک هسته مرکزی به نام Orchestrator مدیریت می‌شوند. دیاگرام زیر جریان اطلاعات را از لحظه ورود درخواست کاربر تا آموزش مدل و ذخیره‌سازی نتایج نشان می‌دهد:



شکل ۲: دیاگرام جریان داده و تعامل اجزای سیستم

۲.۲ توصیف ماژول‌های اصلی

سیستم از سه لایه اصلی تشکیل شده است که هر کدام وظیفه مشخصی را در فایل‌های منبع (src) بر عهده دارند:

۱.۲.۲ لایه داده (Data Layer)

این لایه وظیفه بارگذاری و آماده‌سازی داده‌های پزشکی را بر عهده دارد. کلاس اصلی در این بخش MedicalTextDataset است که فایل‌های CSV را خوانده و با استفاده از Tokenizer مدل انتخاب شده، متن‌ها را به بردارهای عددی تبدیل می‌کند. ویژگی‌های اصلی این لایه عبارتند از:

- مدیریت طول دنباله‌ها با برش (Truncation) و لایه‌گذاری (Padding) تا سقف 512 توکن.
- تبدیل برچسب‌های متنی به تانسورهای PyTorch برای پردازش در مدل.

۲.۲.۲ لایه مدل (Model Factory)

این بخش مسئول نمونه‌سازی از معماری‌های مختلف شبکه عصبی است. بر اساس الگوی طراحی Factory، تابع `get_base_model` می‌تواند مدل‌های زیر را فراخوانی کند:

• BERT (bert-base-uncased)

• BioBERT (dmis-lab/biobert-v1.1)

• RoBERTa (roberta-base)

همچنین ماژول‌های LoRA و Adapter در این لایه پیاده‌سازی شده‌اند تا امکان تزریق لایه‌های تنظیم‌کننده به مدل‌های پایه فراهم شود.

۳.۲.۲ هسته عامل (Agent Core)

این ماژول مغز متفکر سیستم است که منطق تصمیم‌گیری را کنترل می‌کند. کلاس Orchestrator با اتصال به LLM‌های خارجی، استراتژی آموزش را تعیین می‌کند. این هسته از ارائه‌دهندگان زیر پشتیبانی می‌کند:

• Google Gemini: برای تحلیل‌های پیچیده و تولید تنظیمات.

• Groq: برای استنتاج سریع با مدل‌های Llama.

• Perplexity: برای جستجو و استدلال بلادرنگ.

نتایج هر آزمایش شامل دقت (Accuracy) و خطا (Loss) در کلاسی به نام Experience Memory ذخیره می‌شود تا در دوره‌های بعدی (Trials) برای بهبود عملکرد مورد استفاده قرار گیرد.

۳ روش‌های تنظیم دقیق مدل (Fine-Tuning Strategies)

در این پروژه، تمرکز اصلی بر استفاده از روش‌های تنظیم دقیق کارآمد پارامتر (PEFT) است تا بتوان مدل‌های زبانی بزرگ را با هزینه محاسباتی کم برای حوزه پزشکی شخصی‌سازی کرد.

۱.۳ مدل‌های پایه

سیستم از دو مدل پایه اصلی پشتیبانی می‌کند:

- **BERT**: یک مدل عمومی که درک عمیقی از زبان دارد.
- **BioBERT**: نسخه‌ای از BERT که بر روی متون زیست‌پزشکی آموزش دیده و برای طبقه‌بندی بیماری‌ها عملکرد بهتری دارد.

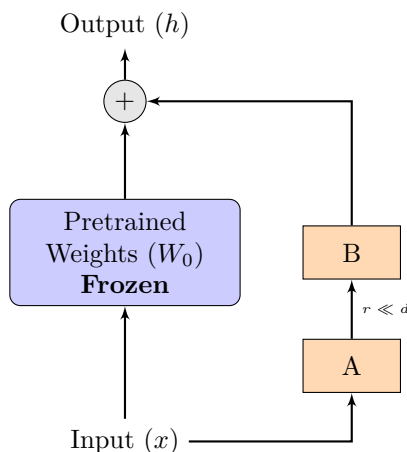
۲.۳ روش LoRA (Low-Rank Adaptation)

روش LoRA یکی از نوآورانه‌ترین روش‌های استفاده شده در این پروژه است. در این روش، ماتریس وزن‌های اصلی مدل پیش‌آموزش دیده (W_0) فریز می‌شود و تغییرات وزن‌ها (ΔW) با استفاده از تجزیه ماتریسی با رنک پایین مدل‌سازی می‌شود. رابطه ریاضی این لایه به صورت زیر است:

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (۱)$$

که در آن $A \in \mathbb{R}^{r \times d}$ و $B \in \mathbb{R}^{d \times r}$ ماتریس‌های قابل آموزشی هستند که r (رنک) بسیار کوچکتر از d است (در کد ما $r = 4$).

شماتیک ساختار پیاده‌سازی شده LoRA در نمودار زیر نمایش داده شده است:



شکل ۳: معماری لایه LoRA: مسیر اصلی فریز شده و مسیر موازی قابل آموزش

۳.۳ روش Adapter

این روش لایه‌های کوچکی را بین لایه‌های اصلی شبکه ترانسفورمر تزریق می‌کند. ساختار آن به صورت Bottleneck طراحی شده است:

۱. **Down-project**: کاهش ابعاد ورودی به یک فضای کوچکتر.
۲. **Non-linearity**: اعمال تابع فعال‌ساز ReLU.
۳. **Up-project**: بازگرداندن ابعاد به اندازه اولیه.

۴.۳ مقایسه پارامترها

در جدول زیر، تفاوت چشمگیر تعداد پارامترهای قابل آموزش در روش‌های پیاده‌سازی شده مشاهده می‌شود:

جدول ۲: مقایسه کارایی روش‌های تنظیم دقیق

وضعیت مدل پایه	پارامترهای قابل آموزش	استراتژی
کاملاً باز	$\sim 110,000,000$	Full Fine-Tuning
فریز شده	$\sim 1,200,000$	Adapter ($r = 4$)
فریز شده	$\sim 300,000$	LoRA ($r = 4$)

۴ چرخه حیات عامل هوشمند (Agentic Workflow)

این بخش توضیح می‌دهد که سیستم چگونه بدون دخالت انسان، فرآیند آزمایش و خطا را مدیریت می‌کند.

۱.۴ تصمیم‌گیری خودکار (Orchestrator)

قلب سیستم، کلاسی به نام Orchestrator است که وظیفه دارد برای هر دور از آزمایش (Trial)، یک پیکربندی جدید پیشنهاد دهد. این ماژول از یک LLM (مانند Gemini) استفاده می‌کند و پرامپتی شامل تاریخچه آزمایش‌های قبلی به آن ارسال می‌کند:

”Maximize Accuracy with limited budget... Suggest config for Trial N.”

مدل زبانی بر اساس نتایج قبلی، پارامترهای جدیدی برای Learning Rate، Batch Size و نوع مدل (BERT/BioBERT) پیشنهاد می‌دهد.

۲.۴ مدیریت حافظه (Experience Memory)

برای اینکه عامل بتواند یاد بگیرد، نیاز به حافظه دارد. کلاس ExperienceMemory نتایج هر آزمایش را ذخیره می‌کند. هر رکورد در حافظه شامل موارد زیر است:

- شناسه آزمایش (Trial ID): شماره نوبت اجرا.
- پیکربندی (Config): پارامترهای استفاده شده.
- معیارها (Metrics): دقت نهایی و میزان خطا.

این اطلاعات در قالب فایل JSON ذخیره شده و در آزمایش‌های بعدی به عنوان ”تجربه” به مدل زبانی تزریق می‌شوند.

۳.۴ مدیریت بودجه (Budget Manager)

از آنجا که منابع محاسباتی محدود است، ماژول BudgetManager محدودیت‌های زیر را اعمال می‌کند:

- حداکثر تعداد آزمایش: عامل مجاز است حداکثر 10 آزمایش انجام دهد تا به بهترین مدل برسد.
- محدودیت زمانی: اگر زمان اجرا از حد مجاز (مثلاً ۲ ساعت) فراتر رود، پروسه متوقف می‌شود.

۵ مجموعه داده و پیش پردازش

در این پژوهش از مجموعه داده Medical Transcriptions (موجود در پلتفرم Kaggle) استفاده شده است. این مجموعه شامل گزارش های متنی تایپ شده پزشکان در تخصص های مختلف است. داده های خام اولیه شامل ستون های متعددی هستند، اما در این پروژه تمرکز اصلی بر روی ستون transcription به عنوان ورودی متن و ستون medical_specialty به عنوان برچسب هدف می باشد. جهت آماده سازی داده ها برای فرآیند آموزش مدل، مراحل زیر مطابق با قطعه کد پیاده سازی شده، انجام گرفته است:

۱. پاک سازی داده ها: ابتدا تمامی ردیف هایی که فاقد مقدار در ستون های متن یا تخصص بودند (Missing Values) حذف شدند تا از ورود نویز به مدل جلوگیری شود.

۲. انتخاب دسته های پرتکرار: توزیع داده ها در دیتاست اصلی نامتوازن است. برای بهبود عملکرد مدل و ایجاد بالانس نسبی، تنها 10 تخصص پزشکی که بیشترین فراوانی را داشتند انتخاب شده و سایر نمونه ها فیلتر شدند.

۳. کدگذاری برچسب ها: با استفاده از روش Label Encoding، نام های متنی تخصص ها (مانند Surgery یا Consult - History and Phy.) به اعداد صحیح بین 0 تا 9 تبدیل شدند.

خروجی نهایی این پیش پردازش در فایل medical_ready.csv ذخیره شده است که ساختار آن در جدول ۳ قابل مشاهده است. این فایل به عنوان ورودی مستقیم به مدل یادگیری ماشین داده می شود.

جدول ۳: ساختار داده های نهایی پس از پیش پردازش

نام ستون	نوع داده	توصیف
text	رشته متنی (String)	متن کامل گزارش پزشکی (رونویسی شده)
label	عدد صحیح (Integer)	شناسه کلاس تخصص (بین 0 تا 9)

۶ تفسیرپذیری و ارزیابی (Evaluation & XAI)

یکی از مهم ترین جنبه های این سیستم، قابلیت اعتماد و شفافیت آن است. در این بخش، معیارهای عددی و روش های بصری برای سنجش عملکرد مدل بررسی می شوند.

۱.۶ معیارهای ارزیابی

برای سنجش کیفیت مدل، از سه معیار اصلی استفاده شده است که در مازول Evaluator پیاده سازی شده اند:

۱. دقت و امتیاز F1: معیار Accuracy درصد کل پیش بینی های صحیح را نشان می دهد. با توجه به عدم تعادل احتمالی در داده های پزشکی، از معیار Macro F1-Score نیز استفاده می شود که میانگین هارمونیک Precision و Recall است:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (2)$$

۲. خطای کالیبراسیون مورد انتظار (ECE): در پزشکی، مهم است که "اطمینان" مدل با "دقت" واقعی آن همخوانی داشته باشد. معیار ECE اختلاف بین اعتماد مدل (Confidence) و دقت واقعی (Accuracy) را اندازه‌گیری می‌کند. هرچه این عدد به 0 نزدیک‌تر باشد، مدل کالیبره‌تر است.

۲.۶ تفسیرپذیری (Explainable AI)

برای جلوگیری از عملکرد "جعبه سیاه"، از روش‌های XAI استفاده شده است:

۱.۲.۶ روش Layer Integrated Gradients

این تکنیک که توسط کتابخانه Captum پیاده‌سازی شده است، گرادیان‌های مدل را نسبت به ورودی‌ها محاسبه می‌کند. به زبان ساده، این روش محاسبه می‌کند که هر کلمه در متن ورودی (مثلاً "تومور" یا "التهاب") چقدر در تصمیم‌گیری نهایی مدل (مثلاً تشخیص "سرطان") تأثیر داشته است.

۲.۲.۶ توضیحات بالینی هوشمند

پس از استخراج کلمات کلیدی توسط گرادیان‌ها، این کلمات به LLM Orchestrator ارسال می‌شوند. سپس مدل زبانی با نقش یک "پاتولوژیست"، یک پاراگراف توضیحی تولید می‌کند که دلیل پزشکی این تشخیص را برای کاربر شرح می‌دهد.

۳.۶ نمودارهای تحلیل عملکرد

خروجی‌های سیستم شامل نمودارهای زیر است:

- Confusion Matrix: برای مشاهده اینکه مدل کدام کلاس‌ها را با هم اشتباه گرفته است.
- ROC Curve: برای بررسی عملکرد مدل در آستانه‌های مختلف تصمیم‌گیری.

۷ پیاده‌سازی و رابط کاربری (Implementation & UI)

۱.۷ تکنولوژی‌های مورد استفاده

این سامانه با استفاده از زبان Python و کتابخانه‌های قدرتمند زیر توسعه یافته است:

- PyTorch: فریم‌ورک اصلی برای ساخت و آموزش شبکه‌های عصبی.
- HuggingFace Transformers: برای بارگذاری مدل‌های زبانی پیش‌آموزش دیده (BERT).
- Captum: ابزار تخصصی برای تفسیرپذیری و استخراج گرادیان‌ها.
- Streamlit: برای طراحی رابط کاربری تعاملی تحت وب.

۲.۷ راهنمای کاربری سامانه

نرم‌افزار دارای دو حالت عملیاتی اصلی است که در نوار کناری (Sidebar) قابل انتخاب هستند:

۱.۲.۷ حالت آموزش (Training Mode)

در این بخش، کاربر می‌تواند پارامترهای عامل هوشمند را تنظیم کند:

- تعیین تعداد آزمایش‌ها (Max Trials) برای جستجوی عامل.
- تنظیم محدودیت‌های Epoch و نرخ یادگیری.
- مشاهده نمودارهای زنده از روند کاهش خطا (Loss) در حین آموزش.

۲.۲.۷ حالت استنتاج (Inference Mode)

در این حالت، کاربر یک متن پزشکی را وارد می‌کند. سیستم پس از تحلیل:

۱. کلاس بیماری و میزان اطمینان را نمایش می‌دهد.
۲. کلمات تاثیرگذار را با رنگ زرد (Highlight) مشخص می‌کند.
۳. توضیح متنی تولید شده توسط هوش مصنوعی را نمایش می‌دهد.

توجه: جزئیات دقیق‌تر، اسکرین‌شات‌ها و راهنمای گام‌به‌گام استفاده از منوها، در یک مستند جداگانه تحت عنوان «راهنمای رابط کاربری» آورده شده است.